Тема 2. Оператор за присвояване. Входно-изходни операции. Съставяне на линейни програми.

Цел: Запознаване с оператора за присвояване и входно-изходните операции, използвани в езика C++ при съставяне на линейни и други програми с различна структура и сложност.

Оператор за присвояване

От предходната тема знаете, че операторът за присвояване (представя се със символа =) може да се използва за присвояване (задаване) на стойност на променлива.

Променливата се записва от лявата страна на =, а от дясната се записва израз:

```
променлива = израз;
```

Пример:

```
total = sum * 1.2;
```

Променливата `total` трябва предварително да е с дефиниран тип. Възможно е дефинирането да се осъществи заедно с инициализацията:

```
double total = sum * 1.2;
```

Запомнете: Не трябва да се бърка операторът за присвояване = с оператора за сравнение „равно ли е“ ==!

Възможно е многократно присвояване, ако в един израз се използват два или повече символа за присвояване =. Многократните присвоявания определят ред на изпълнение на действията отдясно наляво.

Пример:

```
a = b = c = d = 10;
```

Присвояването се извършва в следната последователност:

числото 10 се присвоява на `d`;

стойността на `d` се присвоява на `c`;

стойността на `c` се присвоява на `b`;

стойността на `b` се присвоява на `a`.

Възможно е използването на оператор за присвояване, при който в лявата част променливата е от един тип, а в дясната – константа, променлива или израз от друг тип. В този случай

стойността от дясната страна се преобразува към типа на променливата от лявата страна. Понякога това води до загуба на информация или точност.

Пример:

```
int a;
float b = 5.7833f;

a = b;

std::cout << a;    // Полученият резултат е 5
```

При преобразуване от `float` към `int` дробната част се отхвърля.

Ако се „смесят“ типовете `int` и `char`, символът участва в аритметичния израз чрез съответната си числова стойност в използваната кодировка.

Пример:

```
int a;
char b = 'Z';

a = b;

std::cout << a;
```

При ASCII-съвместима кодировка се извежда кодът на буквата `Z` – числото 90.

Удобно и препоръчително е използването на т. нар. съставни оператори за присвояване, когато променливата от лявата страна се използва и в израза от дясната страна.

Основните съставни аритметични оператори за присвояване са:

```
+=
-=
*=
/=
%=
```

Пример:

```
bonus = bonus + 2;    // bonus += 2;
bonus = bonus - 2;    // bonus -= 2;
bonus = bonus * 2;    // bonus *= 2;
bonus = bonus / 2;    // bonus /= 2;
bonus = bonus % 2;    // bonus %= 2;
```

1. Явно преобразуване на типовете

При неявното преобразуване на типовете данните се преобразуват автоматично съгласно правилата на езика C++.

При явното преобразуване на типовете (typecasting) могат да се използват два формата:

```
(тип_на_данните) израз
```

и

```
тип_на_данните(израз)
```

където `тип_на_данните` може да бъде всеки валиден в C++ тип (`int`, `char`, `float`, `double` и др.), а изразът може да бъде променлива, константа или комбинация от променливи и константи, свързани с операции.

Явната промяна на типа на данните е временна – само за участието им в съответния израз.

Пример:

```
double chastno;
int a, b;

chastno = (double)a / b;
```

или:

```
chastno = double(a) / b;
```

В съвременния C++ за явно преобразуване на типове може да се използва и операторът `static_cast`:

```
chastno = static_cast<double>(a) / b;
```

Променливите `a` и `b` са от цял тип. При операция деление между две цели числа се получава резултат от цял тип и дробната част се губи.

Явното преобразуване към тип `double` временно преобразува стойността на `a` при участието ѝ в израза. Така се получава частно от тип `double`, което се присвоява на променливата `chastno`.

Трите записа:

```
chastno = (double)a / b;
chastno = double(a) / b;
chastno = static_cast<double>(a) / b;
```

водят до реално деление. Формата със `static_cast` е съвременна C++ алтернатива, която ясно показва, че се извършва явно преобразуване на типа.

Самата променлива `a` не променя своя тип и остава от тип `int`.

Входно-изходни операции

Входно-изходната система на C++ работи чрез потоци от информация. Потокът (stream) представлява последователност от данни.

Стандартният вход обикновено е свързан с клавиатурата, а стандартният изход – с екрана на монитора.

Ако потокът е насочен към програмата, той е входен поток (`input stream`), а ако е насочен от програмата навън – изходен поток (`output stream`).

Операторът:

```
std::cin >>
```

се използва за въвеждане на данни от стандартния вход, а:

```
std::cout <<
```

се използва за извеждане на данни на стандартния изход.

Те са дефинирани в заглавния файл:

```
#include <iostream>
```

Забележете, че `std::cout <<` и `std::cin >>` извършват противоположни действия.

Примери:

```
float salary;
```

```
std::cin >> salary;
```

Операторът `std::cin >>` очаква потребителят да въведе стойност за променливата `salary`.

```
std::cout << "Моята заплата е: " << salary;
```

Извеждат се символен низ и реално число.

Често `std::cin` и `std::cout` се използват по двойки. Чрез `std::cout` се извежда подканващо съобщение към потребителя какво трябва да въведе. След това чрез `std::cin` се очаква въвеждането на данните:

```
std::cout << "Каква е вашата заплата? ";
std::cin >> salary;
```

В това упражнение са използвани стандартните средства на C++ за вход и изход `std::cin` и `std::cout`.

Примерни задачи и линейни програми

Задача 2.1.

Какъв ще бъде резултатът от изпълнението на следните операции в C++ програма:

```
6 / 2
6 % 2
29 / 4.0
29.0 / 4
29 / 4
29.0 / 4.0
```

```
29 % 6
```

Примерен вариант на решение:

```
#include <iostream>

int main()
{
    std::cout << 6 / 2;             // Целочислено деление
    std::cout << "\n";

    std::cout << 6 % 2;             // Остатък от целочислено деление
    std::cout << "\n";

    std::cout << 29 / 4.0;          // Деление на реални числа
    std::cout << "\n";

    std::cout << 29.0 / 4 << "\n";  // Деление на реални числа
    std::cout << 29 / 4 << "\n";    // Целочислено деление
    std::cout << 29.0 / 4.0 << "\n";
    std::cout << 29 % 6 << "\n";    // Остатък от целочислено деление

    return 0;
}
```

Получават се следните резултати:

```
3
0
7.25
7.25
7
7.25
5
```

Целочисленото деление се характеризира с операнди от цели типове, при което резултатът също е цяло число и дробната част се отхвърля.

При деление, при което поне единият операнд е реално число, резултатът е реално число.

Обърнете внимание, че управляващият символ за преминаване на нов ред:

```
"\n"
```

може да се постави в същия оператор `std::cout` заедно с извеждането на стойността на израза.

Задача 2.2.

Съставете програма на C++, чрез която се въвеждат вашето име, фамилия и тегло и същите се извеждат в подходящ вид и с пояснителен текст на екрана.

Примерен вариант на решение:

```
/* C++ програма за въвеждане на име, фамилия и тегло
```

```
   и извеждането им в подходящ вид на екрана */

#include <iostream>
#include <string>

int main()
{
    std::string ime;
    std::string familia;
    int teglo;

    std::cout << "\nВъведете своето име: ";
    std::getline(std::cin, ime);

    std::cout << "\nВъведете своята фамилия: ";
    std::getline(std::cin, familia);

    std::cout << "\nВъведете своето тегло: ";
    std::cin >> teglo;

    std::cout << "\n"
              << ime << " " << familia
              << " има тегло " << teglo << " кг.\n";

    return 0;
}
```

В съвременния C++ за работа със символни низове удобно се използва типът:

```
std::string
```

който е дефиниран в заглавния файл:

```
#include <string>
```

За въвеждане на цял ред от символи се използва функцията:

```
std::getline()
```

Например:

```
std::getline(std::cin, ime);
```

въвежда цял ред от стандартния вход и го записва в символния низ `ime`.

По аналогичен начин се формира стойността на променливата `familia`.

Задача 2.3.

Професор провежда изпит в университет. Изпитва 10 студента по учебна дисциплина. Съставете програма на C++, която очаква да въвеждате оценките за 10 студента, намира и извежда средноаритметичната оценка по учебната дисциплина.

Примерен вариант на решение:

```
#include <iostream>

int main()
{
    int oc1, oc2, oc3, oc4, oc5;
    int oc6, oc7, oc8, oc9, oc10;

    double srusp = 0.0;

    std::cout << "\nОценка 1: ";
    std::cin >> oc1;
    srusp = srusp + oc1;

    std::cout << "\nОценка 2: ";
    std::cin >> oc2;
    srusp += oc2;

    std::cout << "\nОценка 3: ";
    std::cin >> oc3;
    srusp += oc3;

    std::cout << "\nОценка 4: ";
    std::cin >> oc4;
    srusp += oc4;

    std::cout << "\nОценка 5: ";
    std::cin >> oc5;
    srusp += oc5;

    std::cout << "\nОценка 6: ";
    std::cin >> oc6;
    srusp += oc6;

    std::cout << "\nОценка 7: ";
    std::cin >> oc7;
    srusp += oc7;

    std::cout << "\nОценка 8: ";
    std::cin >> oc8;
    srusp += oc8;

    std::cout << "\nОценка 9: ";
    std::cin >> oc9;
    srusp += oc9;

    std::cout << "\nОценка 10: ";
    std::cin >> oc10;
    srusp += oc10;

    std::cout << "\nСредният успех по дисциплината е: "
              << srusp / 10;

    return 0;
}
```

Последният оператор `std::cout` може да се запише по друг начин, като се използват изходните манипулатори `std::fixed` и `std::setprecision()`.

Манипулаторът `std::setprecision()` се използва за определяне точността на извеждане на реалните числа.

За извеждане на резултат с два знака след десетичната точка се записва:

```
#include <iomanip>
```

и:

```
std::cout << std::fixed << std::setprecision(2)
          << "\nСредният успех по дисциплината е: "
          << srusp / 10;
```

`std::fixed` задава фиксиран десетичен формат, а:

```
std::setprecision(2)
```

задава два знака след десетичната точка.

В следващ вариант може да дефинирате променливата `srusp` от тип `int` и да използвате явно преобразуване на типа:

```
#include <iostream>

int main()
{
    int oc1, oc2, oc3, oc4, oc5;
    int oc6, oc7, oc8, oc9, oc10;
    int srusp = 0;

    std::cout << "\nОценка 1: ";
    std::cin >> oc1;
    srusp = srusp + oc1;

    // По аналогичен начин се въвеждат останалите оценки.

    std::cout << "\nСредният успех по дисциплината е: "
              << static_cast<double>(srusp) / 10;

    return 0;
}
```

Целочислената променлива `srusp` се преобразува явно, за да се получи резултат реално число.

Същият резултат ще се получи, ако се напише:

```
srusp / 10.0
```

Ако се напише:

```
srusp / 10
```

резултатът от делението ще бъде цяло число, понеже двата операнда са цели числа.

Грешно би било например:

```
double sredno = srusp / 10;
```

Ако `srusp` е от тип `int`, делението `srusp / 10` се извършва първо като целочислено деление. Едва след това резултатът се преобразува към `double`, като загубената дробна част не може да бъде възстановена.

Вместо използването на 10 променливи от цял тип `oc1`, `oc2`, `oc3` и т.н. е възможно да се използва една променлива `oc` и подходящ цикъл. Това ще бъде разгледано при програмите с циклична структура.

Задача 2.4.

Трансформирайте следващите формули в еквивалентни оператори за присвояване в C++ програма:

```
z = (a-b)(a-c)2
```

$$f = \frac{a^2}{b^3}$$

Примерно програмно записване:

```
z = (a - b) * (a - c) * 2;

f = std::pow(a, 2) / std::pow(b, 3);
```

За математически изрази със степени може да се използва стандартната функция:

```
std::pow()
```

дефинирана в заглавния файл:

```
#include <cmath>
```

Примерен вариант на програмно решение:

```
#include <iostream>
#include <cmath>

int main()
{
    double a, b, c;
    double x, z, f;
    double d;

    std::cout
        << "\nВъведете стойности за a, b, c и x "
        << "на един ред, разделени с празен интервал: ";

    std::cin >> a >> b >> c >> x;

    z = (a - b) * (a - c) * 2;

    f = a * a / std::pow(b, 3);

    d = (3.0 + 3.0) / (4.0 + 4.0)
        * (8.0 - x * x)
        * (4.0 + 2.0 - 1.0)
        / ((x - 9.0) * std::pow(x, 3));

    std::cout << "\nz = " << z;
```

```
    std::cout << "\nf = " << f;
    std::cout << "\nd = " << d;

    return 0;
}
```

Обърнете внимание, че при използването на стандартни математически функции като:

```
std::sqrt()
std::pow()
```

е необходимо включването на заглавния файл:

```
#include <cmath>
```

Задача 2.5.

Какъв е изходът от следващата C++ програма с вход и изход?

```
#include <iostream>

int main()
{
    char title[12];

    std::cin.getline(title, 6);

    std::cout << title << "\n";

    return 0;
}
```

Посредством функцията:

```
std::cin.getline(title, 6);
```

могат да бъдат записани най-много **5 въведени символа**, защото едно място е необходимо за завършващия нулев символ `'\0'`.

Тези символи се записват в символния масив `title`.

Изпълнете програмата с различни низове и анализирайте резултатите.

Задача 2.6.

Каква стойност ще има променливата `c` след изпълнението на следната програма:

```
#include <iostream>

int main()
{
    int c;
```

```
    c = 'A' + 5;

    std::cout << c;

    return 0;
}
```

При ASCII-съвместима кодировка числото 5 се прибавя към кода на символа `'A'`, който е 65.

Следователно стойността на `c` е:

```
70
```

В C++ е възможно използването на стойности от тип `char` в аритметични изрази.

Задача 2.7.

Какъв е изходът от следната програма:

```
#include <iostream>

#define X1 a+b
#define X2 X1*X1

int main()
{
    int a = 2;
    int b = 3;

    std::cout << a << "," << b << "," << (X2) << "\n";

    return 0;
}
```

Решение:

Използвайки `X2` от директивата:

```
#define X2 X1*X1
```

препроцесорът извършва текстова замяна.

След разгръщането на макросите изразът практически става:

```
a + b * a + b
```

Поради приоритета на умножението:

```
2 + 3 * 2 + 3 = 11
```

На екрана ще се появи:

```
2,3,11
```

Този пример показва и потенциален проблем при използването на макроси без скоби.

По-безопасен вариант би бил:

```
#define X1 (a+b)
#define X2 (X1*X1)
```

при който резултатът вече ще бъде:

```
25
```

Задача 2.8.

Какъв е типът на резултата в изразите на следващата C++ програма, където `m`, `n`, `k`, `l` са променливи от различен тип?

```
#include <iostream>

int main()
{
    int k = 4;              // Целочислен тип
    float l = 4.5f;         // Реално число с плаваща точка
    char m = 'A';           // Символен тип
    double n = 4.888888;    // Реално число с двойна точност

    std::cout << "\n" << n + l;
    std::cout << "\n" << n + k;
    std::cout << "\n" << k + m;
    std::cout << "\n" << n * m;
    std::cout << "\n" << k + l + m + n;

    return 0;
}
```

Когато в един аритметичен израз участват стойности от различни числови типове, C++ извършва стандартни преобразувания на типовете.

Ако в израз участват например `int` и `double`, целочислената стойност се преобразува към `double` и резултатът е от тип `double`.

При ASCII-съвместима кодировка примерните резултати са приблизително:

```
9.38889
8.88889
69
317.778
78.3889
```

При:

```
k + m
```

към стойността на `k` се прибавя числовият код на символа `'A'`.

Дефиниране с инициализация е възможно и във вида:

```
int k(4);
```

В съвременния C++ често се използва и инициализация със скоби:

```
int k{4};
```

Задача 2.9.

Нека `n` е променлива от тип `int`, а `x` – променлива от тип `double`.

Обяснете разликата между:

```
n = x;
```

и:

```
n = static_cast<int>(x + 0.5);
```

За кои стойности на `x` двата оператора дават един и същ резултат? За кои стойности се различават? Какво става, ако `x` е отрицателно?

За целта съставете и изпълнете с различни стойности на променливата `x` вариант на C++ програма.

Примерен вариант на програмно решение:

```
#include <iostream>

int main()
{
    int n;
    double x;

    std::cout << "\nВъведете реално число: ";
    std::cin >> x;

    n = x;

    std::cout << "\nn = x: "
              << n;

    n = static_cast<int>(x + 0.5);

    std::cout << "\nn = static_cast<int>(x + 0.5): "
              << n;

    return 0;
}
```

При:

```
n = x;
```

преобразуването от `double` към `int` отхвърля дробната част. Преобразуването е в посока към нулата.

Например:

```
x = 2.8  → n = 2
x = -2.8 → n = -2
```

При:

```
n = static_cast<int>(x + 0.5);
```

преди преобразуването към `int` към числото се прибавя `0.5`.

За положителни числа този начин може да се използва за просто закръгляване:

```
x = 2.3 → 2
x = 2.8 → 3
```

Но този подход не работи симетрично за отрицателните числа.

Например:

```
x = -2.3

n = x                    → -2
n = int(x + 0.5)         → -1
```

В съвременния C++ при необходимост от закръгляване до най-близко цяло число може да се използва стандартната математическа функция:

```
std::round(x)
```

от заглавния файл:

```
#include <cmath>
```

Изпълнете програмата с различни положителни и отрицателни стойности на `x` и анализирайте резултатите.

Задача 2.10.

Съставете C++ програма, в която се определят цифрите на въведено от клавиатурата трицифрено число и те се извеждат на екрана.

Примерен вариант на програмно решение:

```
#include <iostream>

int main()
{
    int chislo;
    int edinici, desetici, stotici;
    int temp;

    std::cout << "\nВъведете цяло трицифрено число: ";
```

```
    std::cin >> chislo;

    edinici = chislo % 10;

    // Остатъкът при деление на 10 представлява
    // цифрата на единиците.

    desetici = (temp = chislo / 10) % 10;

    // Операторът за присвояване е използван
    // в аритметичен израз.

    stotici = temp / 10;

    std::cout << "\nЦифра на единици: "
              << edinici;

    std::cout << "\nЦифра на десетици: "
              << desetici;

    std::cout << "\nЦифра на стотици: "
              << stotici;

    return 0;
}
```

Редът:

```
temp = chislo / 10;
```

може да бъде записан отделно:

```
temp = chislo / 10;
desetici = temp % 10;
```

или операторът за присвояване може да бъде използван в израза:

```
desetici = (temp = chislo / 10) % 10;
```

Изпълнете програмата с различни стойности на въвежданото трицифрено число и анализирайте резултатите.

Задача 2.11.

Работник работи в магазин за бяла техника 50 часа седмично. Първите 40 часа се плащат по тарифа 2.50 лв. за 1 час, следващите 5 часа след 40 се плащат един път и половина повече и двойно повече за часовете над 45.

Ако приемем, че данъчният процент е 29, напишете програма, която извежда на екрана брутната заплата, данъците и нетната заплата.

Примерен вариант на програмно решение:

```
#include <iostream>
#include <iomanip>
```

```
int main()
{
    double stavka = 2.50;
    double zaplata, danaci, neto;

    zaplata =
        40 * stavka
        + 5 * 1.5 * stavka
        + 5 * 2 * stavka;

    danaci = zaplata * 0.29;

    neto = zaplata - danaci;

    std::cout << std::fixed << std::setprecision(2);

    std::cout << "\nЗаплатата е: "
              << zaplata << " лв.";

    std::cout << "\nДанъци: "
              << danaci << " лв.";

    std::cout << "\nЧиста сума: "
              << neto << " лв.";

    return 0;
}
```

Резултатите са:

```
Заплатата е: 143.75 лв.
Данъци: 41.69 лв.
Чиста сума: 102.06 лв.
```

За да се поставят два знака след десетичната точка в резултатите за заплатата, данъците и чистата сума, е използвано:

```
std::fixed << std::setprecision(2)
```

Манипулаторът `std::setprecision()` е дефиниран в заглавния файл:

```
#include <iomanip>
```

Задача 2.12.

Напишете C++ програма, която подканва потребителя да въведе число с плаваща точка `x` и след това извежда:

- `sin(x);`
- `cos(x);`
- `tan(x);`
- `sinh(x);`
- десетичния логаритъм от `x`;
- абсолютната стойност на `x`;
- най-близкото цяло число, по-малко или равно на `x`;

- най-близкото цяло число, по-голямо или равно на `x`.

Примерен вариант на програмно решение:

```
#include <iostream>
#include <cmath>

int main()
{
    double x;

    std::cout << "\nВъведете реално число: ";
    std::cin >> x;

    std::cout << "\nСинус от "
              << x << " е " << std::sin(x);

    std::cout << "\nКосинус от "
              << x << " е " << std::cos(x);

    std::cout << "\nТангенс от "
              << x << " е " << std::tan(x);

    std::cout << "\nХиперболичен синус от "
              << x << " е " << std::sinh(x);

    std::cout << "\nДесетичен логаритъм от "
              << x << " е " << std::log10(x);

    std::cout << "\nАбсолютна стойност от "
              << x << " е " << std::fabs(x);

    std::cout
        << "\nНай-близкото цяло число, "
        << "по-малко или равно на "
        << x << ", е " << std::floor(x);

    std::cout
        << "\nНай-близкото цяло число, "
        << "по-голямо или равно на "
        << x << ", е " << std::ceil(x);

    return 0;
}
```

Въведете и изпълнете програмата с различни стойности на въвежданото число `x` и анализирайте резултатите.

Не забравяйте, че при стандартните тригонометрични функции:

```
std::sin(x)
std::cos(x)
std::tan(x)
```

аргументът `x` се задава в радиани.

При използването на:

```
std::log10(x)
```

за реален резултат стойността на `x` трябва да бъде положителна.

Стандартните математически функции са достъпни чрез заглавния файл:

```
#include <cmath>
```

Въпроси и задачи за самостоятелна работа

1. Каква е разликата между унарна и бинарна операция в C++ програма?

2. Каква е разликата при въвеждане на низ от символи чрез:

```
std::cin >>
```

и чрез:

```
std::getline(std::cin, niz)
```

?

3. Какъв е изходът от следната програма?

```
#include <iostream>

#define ANT1 a+a+a
#define ANT2 ANT1-ANT1

int main()
{
    int a = 1;

    std::cout << "Сумата е " << ANT2 << "\n";

    return 0;
}
```

Обяснете получения резултат. Как ще се промени резултатът, ако при дефинирането на макросите се използват скоби?

4. Еквивалентни ли са двата оператора за присвояване?

```
sales *= 4 - factor + bonus;
```

и

```
sales = sales * 4 - factor + bonus;
```

Обосновете отговора си.

5. Еквивалентни ли са двете декларации с инициализация на променливите?

```
double rate = 0.07, time, balance = 0.00;
```

и

```
double rate(0.07), time, balance(0.00);
```

В съвременния C++ е възможна и инициализация с фигурни скоби, например:

```
double rate{0.07}, time, balance{0.00};
```

6. Коректна стойност ли ще получи променливата `v` след изпълнението на израза:

```
v = 6 + (m = 9 - a);
```

ако `v`, `m` и `a` са променливи от цял тип?

Каква стойност ще получи и променливата `m`?

7. Какъв е резултатът от изчислението на всеки от следните изрази?

```
9 % 2 + 1
1 + (10 - (2 + 2))
```

Обяснете последователността на извършване на операциите.

8. Как можете да получите последната цифра на цяло положително число? А първата?

С други думи, ако:

```
int n = 56324;
```

как могат да се получат цифрите:

```
4
```

и

```
5
```

Съставете вариант на C++ програма.

9. Трансформирайте следващата формула в еквивалентен оператор за присвояване в C++ програма:

```
      y * 2 + 5
x = -------------
      14 - 6 / 3
```

Съставете програма, която включва и вход и изход на данните.

10. Напишете C++ програма, която пита потребителя за дължините на две съседни страни на правоъгълник и след това извежда:

- лицето на правоъгълника;
- периметъра на правоъгълника;
- дължината на диагонала на правоъгълника.

За пресмятане дължината на диагонала използвайте Питагоровата теорема и подходяща стандартна математическа функция от заглавния файл:

```
#include <cmath>
```

11. Напишете C++ програма, която пресмята и извежда лицето на разностранен триъгълник със страни 8, 9 и 10.

Използвайте Хероновата формула за намиране лице на триъгълник.

Ако страните на триъгълника са `a`, `b` и `c`, полупериметърът е:

```
s = (a + b + c) / 2
```

а лицето се определя чрез:

```
S = √(s(s-a)(s-b)(s-c))
```

За изчисляване на квадратния корен използвайте:

```
std::sqrt()
```

от заглавния файл:

```
#include <cmath>
```

12. Напишете C++ програма, която извежда всяка от първите 10 степени на числото 2:

2^1, 2^2, 2^3, ..., 2^{10}

Напишете коментари и включете името си в началото на програмата.

Изведете символни низове, които описват всеки от получените резултати.

Първите два реда от резултата на вашата програма трябва да изглеждат така:

```
2 на първа степен е 2
2 на втора степен е 4
```

и т.н.

За пресмятане на степените може да използвате стандартната функция:

```
std::pow()
```

от заглавния файл:

```
#include <cmath>
```